Lab 11 Convolutional Neural Networks

Section 1 Convolution Basics

Given the following 5×5 matrix:

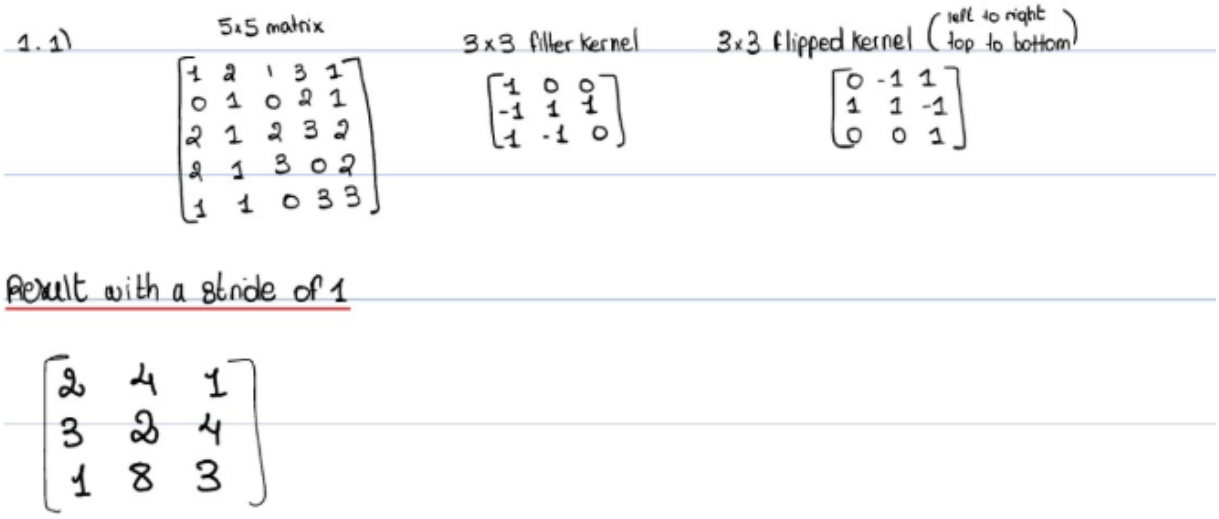
$$\begin{bmatrix} 1 & 2 & 1 & 3 & 1 \\ 0 & 1 & 0 & 2 & 1 \\ 2 & 1 & 2 & 3 & 2 \\ 2 & 1 & 3 & 0 & 2 \\ 1 & 1 & 0 & 3 & 3 \end{bmatrix}$$

and the 3×3 filter (kernel):

$$\begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 1 \\ 1 & -1 & 0 \end{bmatrix}$$

❓ 1.1 Determine the result of the convolution using a stride of 1.

In [1]:
```
from IPython.display import Image, display
display(Image("1.1.png"))
```



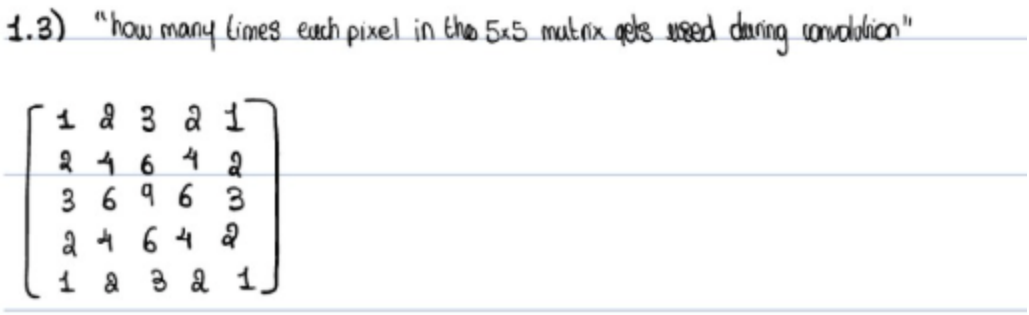
❓ 1.2 Compare the dimensions of the convolved matrix to those of the original matrix. Can you identify any potential issues arising from this difference in size?

✅ Answer: The dimension of the original matrix is 5x5 and the dimension of the convolved matrix is 3x3 (same size as the filter kernel). A potential issue that might arise

from this difference in size is loss of spatial information.

❓ 1.3 Create a 5x5 matrix where each value represents the number of times the corresponding element in the original 5x5 matrix above was used by the filter.

In [2]:
```
from IPython.display import Image, display
display(Image("1.3.png"))
```



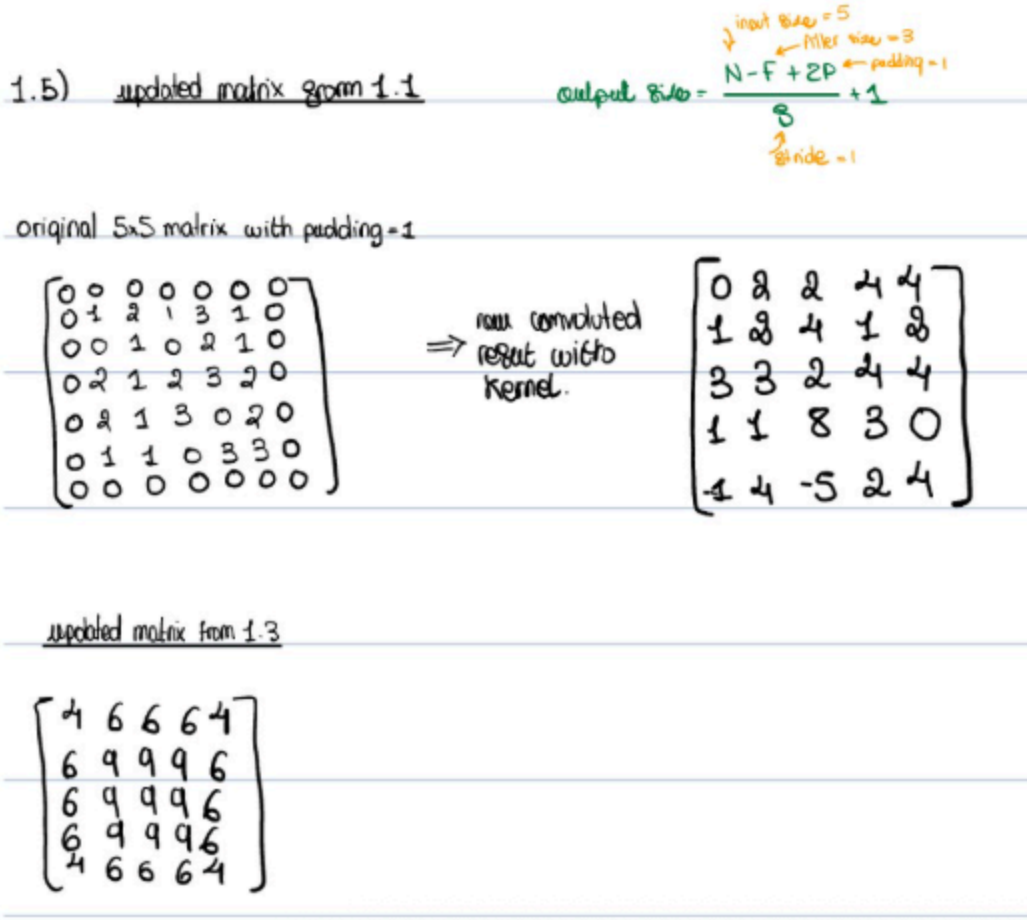
❓ 1.4 What issues could you notice from the matrix above? What methods could you use to address this.

✅ Answer: Some pixels are used more often than others during the convolution. For example, pixels near the borders are used significantly less, which can lead to loss of information in those regions and bias toward the center of the image.

To address this, zero-padding can be used, where extra pixels (typically zeros) are added around the border of the image so that edge pixels are included in more filter applications and the output size is preserved.

❓ 1.5 Apply the approach proposed in 1.4, and list the updated matrices from 1.1 and 1.3 below.

In [3]:
```
from IPython.display import Image, display
display(Image("1.5.png"))
```



❓ From the examples above, try to summarize why we want to use padding.

✅ To preserve the spatial dimensions of the input matrix

✅ To prevent loss of information at the borders (edge pixels)

Section 2: Convolutional Neutral Network (CNN) for Imaging Classification

In this section, you will design your convolutional neural network (CNN) architecture and compare its performance with that of a fully connected feedforward neural network (FFNN) in an image classification task.

In [4]:
```
# load dependencies
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np
import torch
```

Run the code below to make sure you are utilizing GPU resources.

In [5]:
```
# select the appropriate hardware
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)
```

```
cpu
```

2.1 Understand your data

Click here to learn more about the dataset (**CIFAR-10**) that we will be working with today.

In [6]:
```
# load dataset
(X_train, y_train), (X_test,y_test) = datasets.cifar10.load_data()

print(X_train.shape)
print(X_test.shape)
```

```
(50000, 32, 32, 3)
(10000, 32, 32, 3)
```

❓ Explain the shape of X_train and X_test.

✅ Answer: The first column represents the number of images (50,000), the second column represents the image's height (32 pizels), the third column represents the image's width in pixels (32 pizels) and the final column represents the number of color channels (3 for RGB).

2.2 A picture is worth a thousand words

❓ 2.2.1 Please define a dictionary that maps the y values to their corresponding image classes in the **CIFAR-10** dataset.

In [7]:
```
# please write your code below
label_map = {
    0: "airplane",
    1: "automobile",
    2: "bird",
    3: "cat",
    4: "deer",
    5: "dog",
    6: "frog",
    7: "horse",
    8: "ship",
    9: "truck"
}
```

❓ 2.2.2 Please define a visualization function that displays an image given its index in the dataset. The image figsize should be set to (1, 1), and the x-axis should be labeled with its corresponding class.

Use your function to display the first image in the training dataset.

Useful Link: imshow

In [8]:
```
# please write your code below
import matplotlib.pyplot as plt

def show_image(index):
    plt.figure(figsize=(1, 1))
    plt.imshow(X_train[index])
    label = y_train[index][0] #the zero here grabs the value stored at that
    plt.xlabel(label_map[label])
    plt.xticks([])
    plt.yticks([])
    plt.show()

show_image(0)
```

frog

2.3 Build your model

❓ 2.3.1 Apply min-max normalization to your X data to scale values from 0 to 1, and flatten your y values.

Useful Links:

- Normalization in Image Preprocessing
- reshape

In [9]:
```
# please write your code below
# Normalize pixel values (0–255 → 0–1)
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten labels from shape (n,1) → (n,) gives us a list of numbers
y_train = y_train.flatten()
y_test = y_test.flatten()
```

❓ 2.3.2 Build an FFNN with the following specifications:

- The input layer matches the data dimensions.
- The first hidden layer has **3000** neurons with a **ReLU** activation function.
- The second hidden layer has **1000** neurons with a **ReLU** activation function.

- The output layer is set up for this specific multi-class classification task, using a **softmax** activation function.
- Use **Adam** as the optimizer, **sparse categorical crossentropy** as the loss function, and **accuracy** as the evaluation metric.
- Train the model for **10** epochs, and evaluate the model using the test dataset.

Useful Links

- Sequential
- Input
- Flatten
- Dense
- Compile
- Fit
- Evaluate

In [10]:
```
# please write your code below
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense

# Define the FFNN model
model = Sequential()

# Input layer
model.add(Input(shape=(32, 32, 3)))

# Flatten the input to make it compatible with dense layers
model.add(Flatten())

# First hidden layer with 3000 neurons and ReLU activation
model.add(Dense(3000, activation='relu'))

# Second hidden layer with 1000 neurons and ReLU activation
model.add(Dense(1000, activation='relu'))

# Output layer with 10 neurons (for 10 classes) and softmax activation
model.add(Dense(10, activation='softmax'))

# Compile the model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model for 10 epochs
model.fit(X_train, y_train, epochs=10)

# Model evaluation
model.evaluate(X_test, y_test)
```

```
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 49s 31ms/step - accuracy: 0.3258 - loss: 1.89
31
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 49s 31ms/step - accuracy: 0.4033 - loss: 1.66
49
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 51s 32ms/step - accuracy: 0.4325 - loss: 1.58
01
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 49s 31ms/step - accuracy: 0.4527 - loss: 1.52
89
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 49s 31ms/step - accuracy: 0.4633 - loss: 1.49
56
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 49s 31ms/step - accuracy: 0.4747 - loss: 1.46
40
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 49s 31ms/step - accuracy: 0.4874 - loss: 1.43
57
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 49s 32ms/step - accuracy: 0.4940 - loss: 1.41
73
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 49s 32ms/step - accuracy: 0.5036 - loss: 1.39
00
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 49s 32ms/step - accuracy: 0.5086 - loss: 1.37
54
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.4748 - loss: 1.4701
```

Out[10]: `[1.470077395439148, 0.4747999906539917]`

A CNN is primarily composed of convolutional layers that extract features from images, followed by an FFNN that learns the relationships within these features for classification or other tasks.

❓ 2.3.3 Now build a CNN with the following specifications:

- The input layer matches the data dimensions.
- The first convolutional layer has **32** filters of size **3 × 3** with a **ReLU** activation function, followed by **2 × 2** max pooling.
- The second convolutional layer has **64** filters of size **3 × 3** with a **ReLU** activation function, followed by **2 × 2** max pooling.

For the following FFNN section, you can copy and paste your previous work and adjust the settings as needed.

- The hidden layer has **64** neurons with a **ReLU** activation function.
- The output layer is set up for this specific multi-class classification task, using a **softmax** activation function.

- Use **Adam** as the optimizer, **sparse categorical crossentropy** as the loss function, and **accuracy** as the evaluation metric.
- Train the model for **10** epochs, and evaluate the model using the test dataset.

In [11]:
```
# please write your code below
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, De

# Define the CNN model
cnn_model = Sequential()

# Input layer
cnn_model.add(Input(shape=(32, 32, 3)))

# # First convolutional layer with 32 filters of size 3x3 and ReLU activatio
cnn_model.add(Conv2D(32, (3, 3), activation='relu'))
cnn_model.add(MaxPooling2D(pool_size=(2, 2)))

# Second convolutional layer with 64 filters of size 3x3 and ReLU activation
cnn_model.add(Conv2D(64, (3, 3), activation='relu'))
cnn_model.add(MaxPooling2D(pool_size=(2, 2)))

# Dense layer
cnn_model.add(Flatten())
cnn_model.add(Dense(64, activation='relu'))

# Output layer with 10 neurons (for 10 classes) and softmax activation
cnn_model.add(Dense(10, activation='softmax'))

# Compile the model
cnn_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model for 10 epochs
cnn_model.fit(X_train, y_train, epochs=10)

# Model evaluation
cnn_model.evaluate(X_test, y_test)
```

```
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.4791 - loss: 1.459
6
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.6137 - loss: 1.111
1
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.6600 - loss: 0.983
2
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.6862 - loss: 0.901
8
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.7126 - loss: 0.829
6
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.7327 - loss: 0.769
8
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.7476 - loss: 0.721
0
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.7630 - loss: 0.674
9
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.7810 - loss: 0.630
5
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.7931 - loss: 0.594
9
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6963 - loss: 0.9629
```

Out[11]: `[0.9628822207450867, 0.6963000297546387]`

❓ 2.3.4 Compare the true labels, the FFNN-predicted labels, and the CNN-predicted labels for the first 10 images in the test dataset.

Useful Link: argmax

In [12]:
```python
# please write your code below

# Get predictions (probabilities)
ffnn_preds = model.predict(X_test[:10])
cnn_preds = cnn_model.predict(X_test[:10])

# Convert probabilities → predicted class labels
ffnn_labels = np.argmax(ffnn_preds, axis=1)
cnn_labels = np.argmax(cnn_preds, axis=1)

# True labels
true_labels = y_test[:10]

# Print comparison
for i in range(10):
    print(f"Image {i}: True = {true_labels[i]}, FFNN = {ffnn_labels[i]}, CNN
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step
Image 0: True = 3, FFNN = 5, CNN = 3
Image 1: True = 8, FFNN = 9, CNN = 8
Image 2: True = 8, FFNN = 8, CNN = 8
Image 3: True = 0, FFNN = 8, CNN = 0
Image 4: True = 6, FFNN = 4, CNN = 6
Image 5: True = 6, FFNN = 6, CNN = 6
Image 6: True = 1, FFNN = 3, CNN = 1
Image 7: True = 6, FFNN = 4, CNN = 6
Image 8: True = 3, FFNN = 4, CNN = 3
Image 9: True = 1, FFNN = 1, CNN = 1
```

2.4 Understanding CNNs

❓ 2.4.1 Based on your results from sections 2.3.2 and 2.3.3, what are the key advantages of using a CNN architecture over a traditional FFNN for image related tasks? Provide at least 2 advantages

✅ Answer:

- Feature extraction: CNNs automatically learn and extract important features from images, such as edges, textures, and shapes, by using filters that scan over small regions of the image. This allows the model to focus on meaningful patterns instead of raw pixel values.
- Preserves spatial structure: CNNs keep the 2D structure of the image (height and width), so they understand where features are located. This is important because the position of patterns in an image matters, unlike FFNNs which flatten the image and lose this spatial information.

❓ 2.4.2 Explain the role of the pooling layers in our CNN architecture. What benefits do they provide in the image classification process?

✅ Answer:

- Pooling layers reduce the spatial dimensions of feature maps, which lowers computational cost and model complexity.
- They help retain the most important features while discarding less relevant information, improving generalization and reducing overfitting.

❓ 2.4.3 In our CNN architecture, we used 32 filters in the first convolutional layer and 64 in the second. Explain the significance of increasing the number of filters in deeper convolutional layers.

✅ Answer: Increasing the number of filters allows the network to learn a greater variety of features at each layer. Deeper layers use more filters to capture more complex and

abstract patterns in the data, improving the model's ability to recognize intricate structures in images.

In [ ]: